

프로그램 기초
변수

2

수치형 변수

PART 1

1. B / R / D : 해당 변수들은 별도의 선언이 필요치 않는 전역 변수입니다. (비휘발성)
2. LB / LR / LD : 별도의 선언이 필요치 않는 로컬 변수입니다. (휘발성)
3. 사용자 정의 변수는 프로그램 상에서 직접 선언하여 전역/로컬 변수로 정의할 수 있습니다. (휘발성)

변수 이름	자료형		크기	변수 개수
B (LB)	Byte	부호 없는 정수	8bit	0 ~ 255
R (LR)	32Int	부호 있는 정수	32bit	0 ~ 255
D (LD)	Double	부동 소수점 실수	64bit	0 ~ 255
사용자 정의	Bool	논리형 변수	1bit	-
	Byte	부호 없는 정수	8bit	-
	Int	부호 있는 정수	32bit	-
	Float	부동 소수점 실수	32bit	-
	Double	부동 소수점 실수	64bit	-

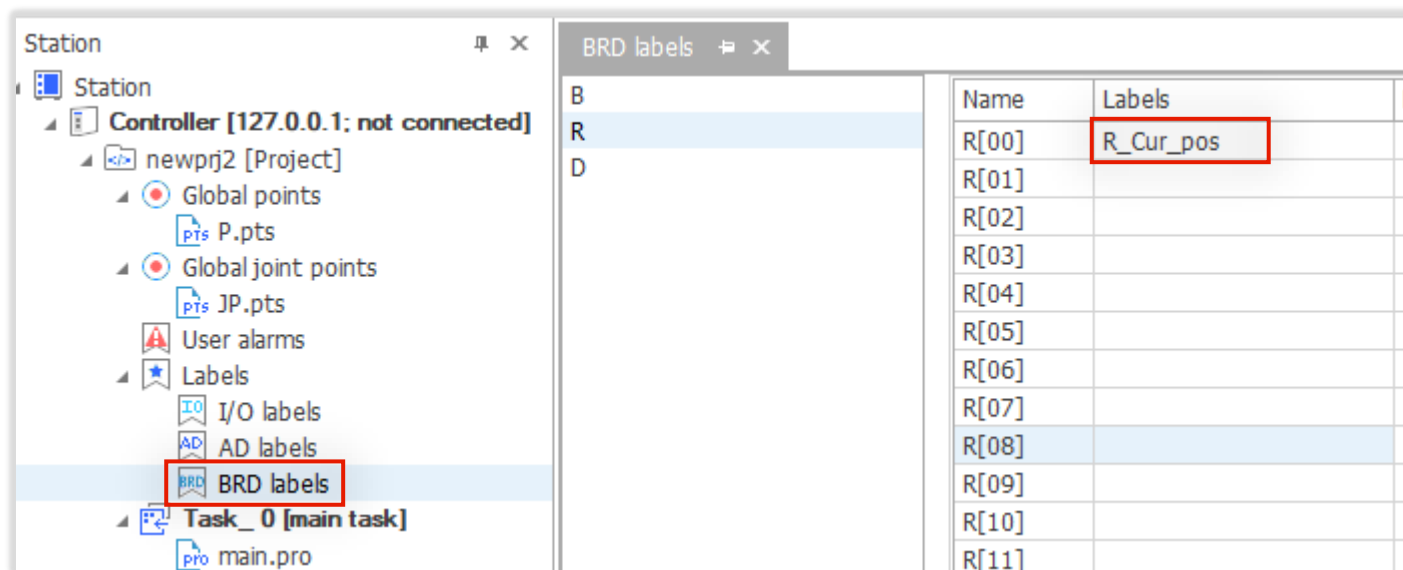
1. B / R / D : 해당 변수는 Label를 사용하여 변수명 혹은 주소로 직접 사용할 수 있습니다. 컨트롤러 OFF에도 값은 저장되며 휘발되지 않습니다.
2. LB / LR / LD : 해당 변수는 Label를 사용할 수 없으며, 주소로 사용되며 하나의 함수 혹은 프로그램에서만 값이 적용됩니다.

변수 이름	자료형		크기	변수 개수
B (LB)	Byte	부호 없는 정수	8bit	0 ~ 255
R (LR)	32Int	부호 있는 정수	32bit	0 ~ 255
D (LD)	Double	부동 소수점 실수	64bit	0 ~ 255

```

Start;
    R[0] = 1;
    R_Cur_pos = 2;
    #R[0] == 2
End;

```



```

Start;
    LR[0] = 0;
    Func_1();
    #LR[0] == 1
End;
Func Func_1()
    LR[0] = 1;
EndFunc;

```

1. 로컬 변수는 Start / End , Func / EndFunc 사이에 선언하며, 함수 혹은 프로그램 간 공유가 불가능합니다.
2. 글로벌 변수는 Start 문구 상단에 선언하며, 함수 혹은 프로그램 간 공유가 가능합니다.

변수 이름	자료형		크기	변수 개수
사용자 정의	Bool	논리형 변수	1bit	-
	Byte	부호 없는 정수	8bit	-
	Int	부호 있는 정수	32bit	-
	Float	부동 소수점 실수	32bit	-
	Double	부동 소수점 실수	64bit	-

```

1  Global Int iGlobal_var;
2  Start;
3      Int local_var = 1;
4      Bool test_var;
5      Func_1();
6      Print local_var;
7      Print iGlobal_var;
8      #local_var = 1
9      #iGlobal_var = 10
10 End;
11 Func Func_1()
12     Int local_var = 5;
13     iGlobal_var = 10;
14 EndFunc;
15

```

문자열 변수

PART 2

1. Str : 해당 변수는 별도의 선언이 필요치 않는 전역 변수입니다. (비휘발성)
2. 사용자 정의 변수는 프로그램 상에서 직접 선언하여 전역/로컬 변수로 정의할 수 있습니다. (휘발성)

변수 이름	자료형		크기	변수 개수
Str	String	문자열	-	0 ~ 255
사용자 정의	String	문자열	-	-

```

1 Start;
2     Str[0] = "Str var text"
3 End;
```

```

1 Global String strGlobal_var;
2 Start;
3     String local_var = "Test text";
4     Func_1();
5     Print local_var;
6     Print strGlobal_var;
7     #local_var == Test text
8     #strGlobal_var == Global text
9 End;
10 Func Func_1()
11     String local_var = "Func text";
12     strGlobal_var = "Global text";
13 EndFunc;
```

I/O 변수

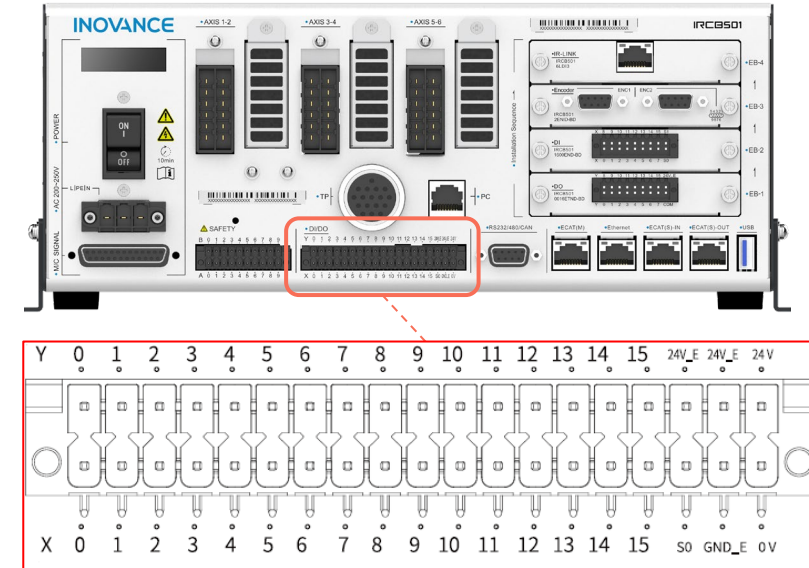
PART 3

I/O 변수 종류 - Standard I/O

1. Standard I/O는 하드웨어 배선 DIO를 의미합니다.
2. In/Out 0 ~ 127 까지 주소에 할당되어 있습니다.
3. Label을 사용하여 변수명으로 사용할 수 있습니다.

```
1 Start;  
2   If In[0] == ON  
3     Set Out[0],ON;  
4     Out[1] = ON;  
5   EndIf;  
6 End;
```

```
1 Start;  
2   If xInput_1 == ON  
3     Set yOutput_1,ON;  
4     yOutput_2 = ON;  
5   EndIf;  
6 End;
```



I/O labels			
Standard I/O			
Input			
Bit	1	In[000]	xInput_1
Byte	2	In[001]	
Word	3	In[002]	
Output			
Bit	4	In[003]	
Byte	5	In[004]	
Word	6	In[005]	
	7	In[006]	
	8	In[007]	

I/O labels			
Standard I/O			
Input			
Bit	1	Out[000]	yOutput_1
Byte	2	Out[001]	
Word	3	Out[002]	
Output			
Bit	4	Out[003]	
Byte	5	Out[004]	
Word	6	Out[005]	
	7	Out[006]	
	8	Out[007]	

1. Fieldbus slave I/O는 통신 전용 (로봇이 슬레이브) DIO를 의미합니다.
2. In/Out 512 ~ 4607 까지 주소에 할당되어 있습니다.
3. Label을 사용하여 변수명으로 사용할 수 있습니다.

```
1 Start;  
2   If xComm_input_1 == ON  
3     Set xComm_output_1, ON;  
4     xComm_output_2 = ON;  
5   EndIf;  
6 End;
```

I/O labels			
▶ Standard I/O			
└ Fieldbus slave I/O			
└ Input			
Bit	Index	Name	Labels
	1	In[512]	xComm_input_1
	2	In[513]	
	3	In[514]	
	4	In[515]	
	5	In[516]	
	6	In[517]	
	7	In[518]	
	8	In[519]	
└ Output			
Bit			
Byte			
Word			

I/O labels			
▶ Standard I/O			
└ Fieldbus slave I/O			
└ Input			
Bit	Index	Name	Labels
	1	Out[512]	xComm_output_1
	2	Out[513]	xComm_output_2
	3	Out[514]	
	4	Out[515]	
	5	Out[516]	
	6	Out[517]	
	7	Out[518]	
	8	Out[519]	
└ Output			
Bit			
Byte			
Word			

1. Fieldbus slave I/O는 통신 전용 (로봇이 마스터) DIO를 의미합니다.
2. In/Out 6656 ~ 8703 까지 주소에 할당되어 있습니다.
3. Label을 사용하여 변수명으로 사용할 수 있습니다.

```
1 Start;  
2   If xComm_input_1 == ON  
3     Set xComm_output_1, ON;  
4     xComm_output_2 = ON;  
5   EndIf;  
6 End;
```

I/O labels		
▶ Standard I/O		
▶ Fieldbus slave I/O		
▶ Fieldbus master I/O		
▾ Memory I/O		
▾ Input		
Bit		
Byte		
Word		
▾ Output		
Bit		
Byte		
Word		
Index	Name	Labels
1	In[12800]	xMemory_input
2	In[12801]	
3	In[12802]	
4	In[12803]	
5	In[12804]	
6	In[12805]	
7	In[12806]	
8	In[12807]	
9	In[12808]	
10	In[12809]	

I/O labels		
▶ Standard I/O		
▶ Fieldbus slave I/O		
▶ Fieldbus master I/O		
▾ Memory I/O		
▾ Input		
Bit		
Byte		
Word		
▾ Output		
Bit		
Byte		
Word		
Index	Name	Labels
1	Out[12800]	xMemory_output_1
2	Out[12801]	xMemory_output_2
3	Out[12802]	
4	Out[12803]	
5	Out[12804]	
6	Out[12805]	
7	Out[12806]	
8	Out[12807]	
9	Out[12808]	
10	Out[12809]	

1. Memory I/O는 로봇 내부에서 사용하는 DIO를 의미합니다.
2. 메모리 IO 특징은 Input을 로봇 프로그램 내에서 직접 제어할 수 있습니다.
3. In/Out 12800 ~ 13823 까지 주소에 할당되어 있습니다.
4. Label을 사용하여 변수명으로 사용할 수 있습니다.

```
1 Start;  
2   If xMemory_input_1 == ON  
3     Set xMemory_output_1, ON;  
4     xMemory_input_2 = ON;  
5   EndIf;  
6   #xMemory_input_2 : Program start  
7 End;
```

I/O labels			
▶ Standard I/O			
▶ Fieldbus slave I/O			
▶ Fieldbus master I/O			
▲ Memory I/O			
▲ Input			
Bit			
Byte			
Word			
▲ Output			
Bit			
Byte			
Word			
Index	Name	Labels	
1	In[12800]	xMemory_input_1	
2	In[12801]	xMemory_input_2	
3	In[12802]		
4	In[12803]		
5	In[12804]		
6	In[12805]		
7	In[12806]		
8	In[12807]		
9	In[12808]		
10	In[12809]		

I/O labels			
▶ Standard I/O			
▶ Fieldbus slave I/O			
▶ Fieldbus master I/O			
▲ Memory I/O			
▲ Input			
Bit			
Byte			
Word			
▲ Output			
Bit			
Byte			
Word			
Index	Name	Labels	
1	Out[12800]	xMemory_output_1	
2	Out[12801]	xMemory_output_2	
3	Out[12802]		
4	Out[12803]		
5	Out[12804]		
6	Out[12805]		
7	Out[12806]		
8	Out[12807]		
9	Out[12808]		
10	Out[12809]		

1. I/O 변수는 Bit외에도 Byte, Word, Int, Double 등으로 변환할 수 있습니다.
2. 32bit, 64bit 표현은 2개, 4개의 Word를 연속으로 사용하여 표현합니다.

#1. I/O 변수 값 쓰기

```
Out[0] = ON;
Set Out[0],ON;
```

#In[12800~13823]만 입력 값 제어 가능

```
In[12800] = ON;
```

#2. I/O Byte/Word로 값 쓰기

```
OutB[0] = 3; #Out[0~7]의 값 제어
OutW[0] = 15; #Out[0~15]의 값 제어
```

#3. I/O Byte/Word의 Bit 제어

```
OutB[0].bit[0] = ON; #Out[0] = ON
OutW[0].bit[15] = ON; #Out[15] = ON
```

#4. I/O 데이터 타입 변경

```
#OutW[0]을 32bit인 Int로 사용한다.
OutW[0].Int = 1000000;
```

#InW[512]를 64bit 실수형으로 D[0] 변수에 저장한다.

```
D[0] = InW[512].Double;
```

	Bus input	Bus output	Byte register (input as example)	Word register (input as example)	
(16 bits, 2 bytes, 1 word)	In0-In15	Out0-Out15	InB0-InB1	InW0	Standard I/O
(112 bits, 14 bytes, 7 words)	In16-In127	Out16-Out127	InB2-InB15	InW1-InW7	Expansion I/O
(384 bits, 48 bytes, 24 words)	In128-In511	Out128-Out511	InB16-InB63	InW8-InW31	Reserved
(4096 bits, 512 bytes, 256 words)	In512-In4607	Out512-Out4607	InB64-InB575	InW32-InW287	Fieldbus I/O - slave
(13824 bits, 1728 bytes, 864 words)	In4608-In6655	Out4608-Out6655	InB576-InB831	InW288-InW415	Reserved
(8192 bits, 1024 bytes, 512 words)	In6656-In10751	Out6656-Out10751	InB832-InB1343	InW416-InW671	Fieldbus I/O - master
(1024 bits, 128 bytes, 64 words)	In10752-In12799	Out10752-Out12799	InB1344-InB1599	InW672-InW799	Reserved
(8192 bits, 1024 bytes, 512 words)	In12800-In13823	Out12800-Out13823	InB1600-InB1727	InW800-InW863	Memory I/O
(1024 bits, 128 bytes, 64 words)					

위치 변수

PART 4

- ```
Start;
 Movj P[3],V[50],Fine,Tool[0],Wobj[0];
 MovAbsJ JP[0],V[30],Z[0],Tool[0];
```

| Point |        | Position info |          |       |       | Arm parameters |          |     |     | External axis |       |       |       |       |       |
|-------|--------|---------------|----------|-------|-------|----------------|----------|-----|-----|---------------|-------|-------|-------|-------|-------|
| ID    | Labels | X             | Y        | Z     | A     | L/R            | J4 range | [2] | [3] | E1            | E2    | E3    | E4    | E5    | E6    |
| 0     |        | 50.000        | 400.000  | 0.000 | 0.000 | Left hand      | -90°~0°  | 0   | 0   | 0.000         | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| 1     |        | 550.000       | 0.000    | 0.000 | 0.000 | Left hand      | 0°~90°   | 0   | 0   | 0.000         | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |
| 2     |        | 50.029        | -399.977 | 0.000 | 0.000 | Left hand      | 90°~180° | 0   | 0   | 0.000         | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 |

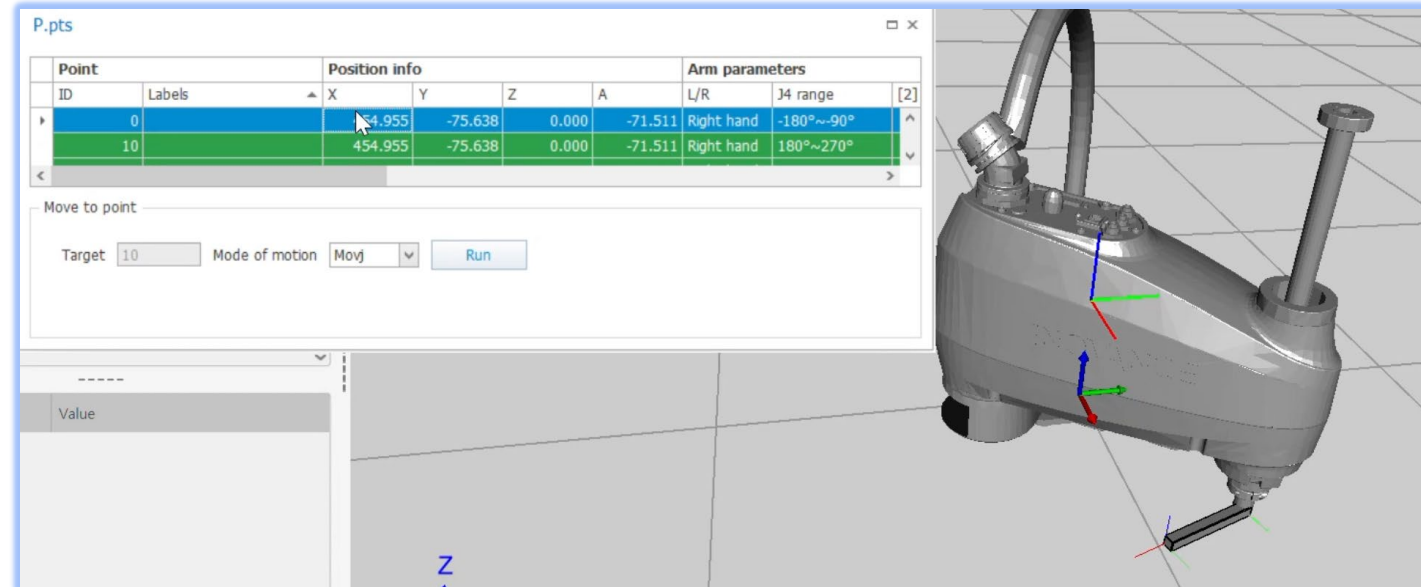
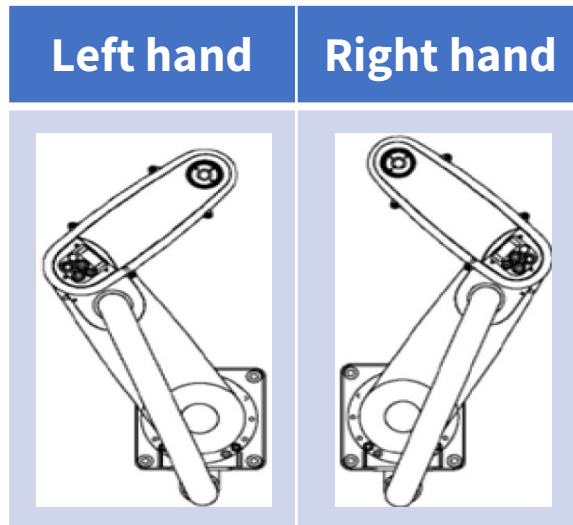
[illegible]



1. 로봇은 해당 위치에 여러 방향(자세)으로 도달할 수 있기 때문에 특정 방향(자세)을 지정해야 합니다.
2. 일반 SCARA 로봇은 2개의 암 파라미터(왼/오른손 자세, J4 회전 각도)를 사용합니다.
3. 왼/오른손 자세는 SCARA 로봇의 1축이 왼쪽 or 오른쪽 자세를 의미합니다.
4. J4 회전 각도는 J4 위치의 회전 각도를 의미합니다.

| Position info |         |       |         | Arm parameters |            |
|---------------|---------|-------|---------|----------------|------------|
| X             | Y       | Z     | A       | L/R            | J4 range   |
| 454.955       | -75.638 | 0.000 | -71.511 | Right hand     | -180°~-90° |
| 454.955       | -75.638 | 0.000 | -71.511 | Right hand     | 180°~270°  |

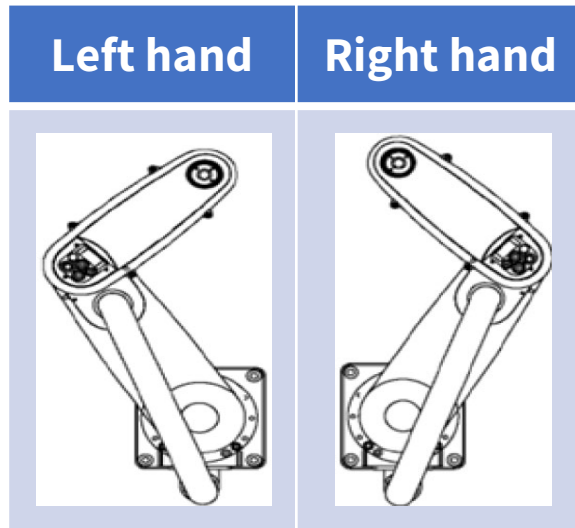
| P.pts |        | JP.pts        |        |       |          |
|-------|--------|---------------|--------|-------|----------|
| Point |        | Position info |        |       |          |
| ID    | Labels | J1            | J2     | J3    | J4       |
| 0     |        | -37.872       | 90.560 | 0.000 | -124.199 |
| 1     |        | -37.872       | 90.560 | 0.000 | 235.801  |





1. 로봇은 해당 위치에 여러 방향(자세)으로 도달할 수 있기 때문에 특정 방향(자세)을 지정해야 합니다.
2. 천장형 SCARA 로봇은 4개의 암 파라미터(왼/오른손 자세, J1, 2, 4 회전 각도)를 사용합니다.
3. 왼/오른손 자세는 SCARA 로봇의 1축이 왼쪽 or 오른쪽 자세를 의미합니다.
4. J4 회전 각도는 J4 위치의 회전 각도를 의미합니다.
5. [2]는 J1의 회전 각도를 의미합니다.
6. [3]은 J2의 회전 각도를 의미합니다.

| Arm parameters |          |     |     |
|----------------|----------|-----|-----|
| L/R            | J4 range | [2] | [3] |
| Right hand     | 0°~90°   | 0   | -2  |
| Right hand     | 0°~90°   | 0   | -2  |
| Right hand     | 0°~90°   | -1  | -2  |
| Right hand     | 0°~90°   | 0   | -2  |



| P.pts |        | JP.pts        |        |       |          |  |
|-------|--------|---------------|--------|-------|----------|--|
| Point |        | Position info |        |       |          |  |
| ID    | Labels | J1            | J2     | J3    | J4       |  |
| 0     |        | -37.872       | 90.560 | 0.000 | -124.199 |  |
| 1     |        | -37.872       | 90.560 | 0.000 | 235.801  |  |

| 회전 각도 표 (J1, 2) |             |            |         |        |          |           |           |
|-----------------|-------------|------------|---------|--------|----------|-----------|-----------|
| -4              | -3          | -2         | -1      | 0      | 1        | 2         | 3         |
| -360 ~ -270     | -270 ~ -180 | -180 ~ -90 | -90 ~ 0 | 0 ~ 90 | 90 ~ 180 | 180 ~ 270 | 270 ~ 360 |

1. 로봇은 해당 위치에 여러 방향(자세)으로 도달할 수 있기 때문에 특정 방향(자세)를 지정해야 합니다.
2. 6-Axis 로봇은 4개의 암 파라미터(J1, 4, 6, 각 관절 자세)를 사용합니다.
3. [1], [2], [3]은 각각 J1, J4, J6의 회전 각도를 의미합니다. (회전 각도 표 참조)
4. [4]는 각 관절 자세를 의미합니다. (관절 자세 표 참조)

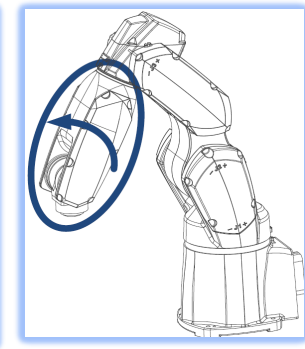
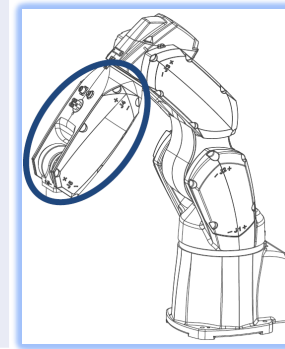
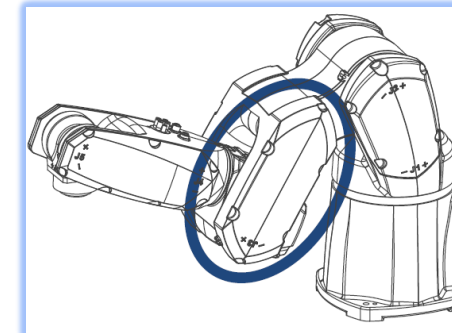
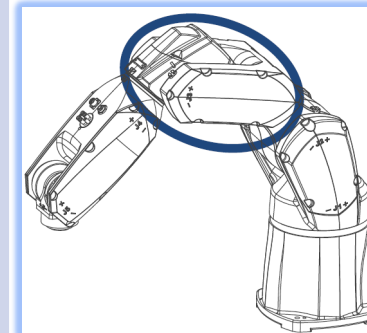
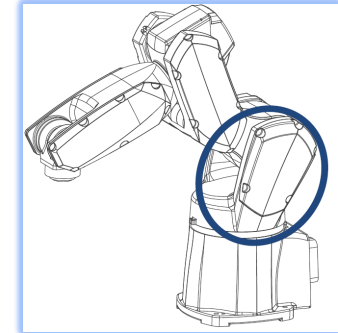
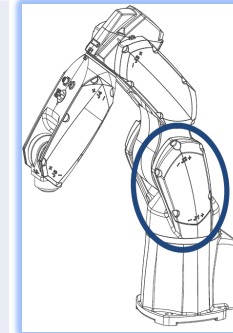
| Point |         | Position info |       |       |       |       |       | Arm parameters |     |     |     | E |
|-------|---------|---------------|-------|-------|-------|-------|-------|----------------|-----|-----|-----|---|
| ID    | Labels  | X             | Y     | Z     | A     | B     | C     | [1]            | [2] | [3] | [4] | E |
| ▶ 100 | pP1_app | 0.000         | 0.000 | 0.000 | 0.000 | 0.000 | 0.000 | 0              | 0   | 0   | 0   |   |

| 회전 각도 표 (J1, 4, 6) |             |            |         |        |          |           |           |
|--------------------|-------------|------------|---------|--------|----------|-----------|-----------|
| -4                 | -3          | -2         | -1      | 0      | 1        | 2         | 3         |
| -360 ~ -270        | -270 ~ -180 | -180 ~ -90 | -90 ~ 0 | 0 ~ 90 | 90 ~ 180 | 180 ~ 270 | 270 ~ 360 |

1. [4]는 각 관절 자세를 의미합니다.

관절 자세 표

| 관절  | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
|-----|----|----|----|----|----|----|----|----|
| 허리  | 전방 | 전방 | 전방 | 전방 | 후방 | 후방 | 후방 | 후방 |
| 팔꿈치 | 상승 | 상승 | 하강 | 하강 | 상승 | 상승 | 하강 | 하강 |
| 손목  | 향상 | 향하 | 향상 | 향하 | 향상 | 향하 | 향상 | 향하 |



1. PR 변수는 P 변수와 비슷하게 위치 값을 저장하는 변수이지만 로봇 움직임 특성을 저장하지 않기 때문에 이동 명령어의 변수로 사용할 수 없습니다.
2. PR 변수는 로봇을 이동시키기 위한 변수가 아닌 위치를 변경하거나 계산하는 용도로 사용됩니다.
3. 0~255까지 총 256개의 PR 변수를 사용할 수 있습니다.

| Name  | Value |
|-------|-------|
| PR[0] |       |
| X     | -150  |
| Y     | 130   |
| Z     | 0     |
| A     | 10    |
| B     | 0     |
| C     | 0     |

## 두 P 변수의 차를 계산하기

```
PR[0] = Msft(P[1],P[2]); #PR[0]에 P[2] - P[1]의 값을 저장한다.
P[1] = X:200, Y:150, Z:-10
P[2] = X:50, Y:20, Z:-30
PR[0] = X:-150, Y:-130, Z:-20
```

## P[1] + PR[1] 위치로 이동하기

```
#P[1]위치에 PR[1]값을 오프셋한 위치로 이동한다.
#PR[1] = X:0, Y:50, Z:-20
PR[1] = (0,50,-20,0,0,0);
Movj Offset(P[1],PR[1]),V[30],Z[0],Tool[0],Wobj[0];
P[1] = X:200, Y:-110, Z:-10
이동 위치 = X:200, Y:-60, Z:-30
```

# 변수명 표기법

## PART 5

1. 데이터 타입을 명확하게 확인하기 위하여 변수명(Label)에 선두에 데이터 타입을 기입하는 헝가리안 표기법을 권장합니다.
2. B, R, D 변수는 일반 표기법과 다르게 대문자와 \_ 가 포함됩니다.

```

1 Start;
2 If xMemory_input_1 == ON
3 Set xMemory_output_1,ON;
4 xMemory_input_2 = ON;
5 EndIf;
6 #xMemory_input_2 : Program start
7 End;
```

| 변수                | 단위                | 타입         | 변수명                                                |
|-------------------|-------------------|------------|----------------------------------------------------|
| IN / Out          | Bit               | IO         | xVar / yVar                                        |
| InB / OutB        | Byte              | IO byte    | xbVar / ybVar                                      |
| InW/OutW          | Word              | IO Word    | xwVar / ywVar                                      |
| B                 | Byte              | 글로벌 B      | B_Var_var2                                         |
| R                 | Integer           | 글로벌 R      | R_Var_var2                                         |
| D                 | Double            | 글로벌 D      | D_Var_var2                                         |
| Bool              | Bit               | 글로벌 사용자 정의 | bVar_var2                                          |
| Byte              | Byte              | 글로벌 사용자 정의 | byVar_var2                                         |
| Int               | Integer           | 글로벌 사용자 정의 | iVar_var2                                          |
| Float             | Float             | 글로벌 사용자 정의 | fVar_var2                                          |
| Double            | Double            | 글로벌 사용자 정의 | dVar_var2                                          |
| String            | String            | 글로벌 사용자 정의 | strVar_var2                                        |
| Array             | Array             | 글로벌 사용자 정의 | aiVar (Array Int data)<br>afVar (Array Float data) |
| Struct / Instance | Struct / Instance | 사용자 정의     | stVar                                              |
| Local             | Local             | 로컬 사용자 정의  | 모든 로컬 변수는 데이터 타입을 표기하지 않습니다.                       |

# INOVANCE

Forward, Always Progressing